

Keywords: token compression • context distillation • LLM efficiency • attention mechanisms

Progressive Cramming: Reliable Token Compression and What It Reveals

A Token-by-Token Compression Protocol Uncovers a Fundamental Capacity Limit in LLM Embedding Space—and Traces Downstream Task Degradation to Destructive Early-Layer Attention Interactions

By Dmitrii Tarasov, Timofei Lashukov, Elizaveta Goncharova, Andrey Kuznetsov

arXiv:2607.21231 • July 23, 2026

Token cramming—compressing long input prefixes into a small set of learned continuous embeddings—promises to let LLMs reference long contexts at the cost of a handful of extra tokens. Prior work reports near-perfect reconstruction (99%+), but fixes both the token budget and the accuracy threshold, leaving a crucial ambiguity: is residual error an optimization shortfall, or a fundamental limit of the embedding space? This paper answers with **progressive cramming**, a protocol that grows the compression target token-by-token until failure, revealing hard content-dependent limits that optimization cannot breach.

AT A GLANCE

Problem: Existing token cramming evaluates at fixed budgets and thresholds, conflating optimization failure with fundamental compression limits.

Why it matters: Systems relying on token cramming for long-context compression may degrade silently at inference; knowing true compression horizons is essential for safe deployment.

Method: Progressive cramming grows target prefix token-by-token, stopping at reconstruction failure; causal attention-knockout interventions locate the degradation source.

Result: Crammed embeddings cause 4–9 pp accuracy drops on multiple-choice tasks and near-total collapse on generative tasks, traced to early-layer attention interactions.

Evidence: Compression horizons range from 28–74% of prefix length depending on content, revealing fundamental limits independent of optimizer tuning.

The canonical approach to long-context LLM inference is to extend the context window—but this incurs quadratic attention cost. Token cramming offers an alternative: compress a prefix of N tokens into $M \ll N$ learned embedding vectors, insert them into the context, and let the model proceed as if the prefix were present. Studies have shown these crammed embeddings can achieve > 99% token-level reconstruction accuracy, fueling optimism about practical deployment.

This paper asks a harder question: does reconstruction accuracy predict task performance? And is there a hard ceiling on how many tokens can be crammed, independent of how long you optimize? The answers are “no” and “yes”, respectively, with implications for any deployment of cramming-based compression.

Why Existing Approaches Fall Short

Standard cramming evaluates quality at a fixed compression ratio (e.g., compress 512 tokens into 32 embeddings) and declares success when reconstruction accuracy exceeds 99%. This methodology has two gaps:

Conflation of failure modes. If reconstruction fails at a given ratio, it is unclear whether (a) more optimization steps would succeed or (b) the information exceeds the embedding’s representational capacity. Distinguishing these requires a protocol that rules out optimization failure as an explanation.

Reconstruction $\neq$ task performance. A crammed embedding that faithfully reconstructs a prefix token-by-token may still disrupt downstream task performance if the embedding interferes with the model’s internal processing—even when the original prefix is also provided in context.

Core Mechanism

Progressive cramming resolves the first gap by progressively growing the target prefix from length 1 to length N , applying a fixed optimization budget at each step. At length n , it finds the best embedding for the first n tokens. At some critical length n^* , optimization fails to achieve a fixed reconstruction threshold within the budget. Because the budget is held constant throughout, failure at n^* cannot be attributed to insufficient compute—it reflects a genuine representational limit.

A key structural finding is that successful progressive trajectories $\{e_1^*, e_2^*, \dots, e_{n^*}^*\}$ lie in a low-dimensional manifold in embedding space. PCA on these trajectories shows 80% of variance explained by 5 principal components—far fewer than the ambient embedding dimension of 4096—suggesting that token cramming exploits a structured low-dimensional code in the embedding space.

The Big Picture

Technical Formulation

Let the LLM f_θ be frozen with token vocabulary $\mathcal{V}$. Given prefix $x = (x_1, \dots, x_N) \in \mathcal{V}^N$, the cramming objective finds an embedding $e \in \mathbb{R}^d$:

$$e_n^* = \arg \min_{e \in \mathbb{R}^d} \mathcal{L}_{\text{recon}}(e, x_{1:n}), \quad \mathcal{L}_{\text{recon}} = \sum_{i=1}^n \ell_{\text{CE}}(f_\theta(e, x_{<i}), x_i)$$

subject to an optimization budget of B gradient steps. Progressive cramming solves this sequence for $n = 1, 2, \dots, N$, stopping at:

$$n^* = \max\{n : \mathcal{L}_{\text{recon}}(e_n^*, x_{1:n}) \leq \tau\}$$

The **compression horizon** is $\rho = n^*/N \in (0, 1]$: the fraction of the prefix representable within budget B .

Learning or Inference Procedure

After establishing compression horizons, the paper tests whether crammed embeddings e_n^* degrade task performance by prepending them to contexts where the full original prefix is also available:

1. Insert e^* as a soft prompt token before the original prefix.
2. Query the model on multiple-choice and open-ended generation tasks.
3. Compare accuracy to a baseline where no crammed embedding is inserted (original prefix only).

To locate the source of degradation, causal attention-knockout interventions systematically ablate attention edges between e^* and each layer’s query/key/value projections, measuring which ablation most recovers task performance.

What the Guarantee Says

There is no guarantee that a crammed embedding at the compression horizon n^* will not degrade task performance. The results show the opposite: even maximally compressed embeddings (at the representational limit) cause consistent accuracy drops. The horizon tells practitioners how much of a prefix the embedding *encodes*—but not how much it *transmits usefully to the model at inference time*.

Experimental Findings

Experiments use GPT-2-XL (1.5B), LLaMA-2-7B, and Mistral-7B as backbone models, with prefixes from C4 (general text), PubMed (medical), and HumanEval (code).

Task Type	No Crammed Embed.	+ Crammed Embed.
MMLU (4-choice)	63.4	56.7
HellaSwag (4-choice)	74.1	67.8
OpenGen (correctness)	51.3	18.2
Code completion	42.7	11.4

Compression horizons (fraction of 512-token prefixes representable): General text: 68%, Medical text: 55%, Code: 28%. Code is hardest to compress, consistent with the view that low-entropy, syntactically precise sequences exhaust embedding capacity fastest.

Ablations and Interpretation

Attention knockout: Ablating attention edges from e^* in layers 1–4 recovers 61% of the multiple-choice accuracy drop. Ablating in layers 12–16 recovers only 8%. Early layers are the primary site of destructive interaction.

Prefix vs. no prefix: Inserting e^* with the original prefix still degrades performance. Inserting e^* without the original prefix is catastrophic. This rules out the hypothesis that the embedding “blocks” the prefix from the model; rather, the embedding actively interferes with early attention distributions.

Budget sensitivity: Doubling B from 10^3 to 2×10^3 steps increases the compression horizon by only 3–6%, confirming that the horizon is not a function of insufficient optimization.

Dimension sensitivity: Increasing embedding dimension from $d = 4096$ to $d = 8192$ raises horizons by 11% on average, suggesting that embedding dimensionality is a partial bottleneck, though the gains plateau rapidly.

Reference

Dmitrii Tarasov, Timofei Lashukov, Elizaveta Goncharova, Andrey Kuznetsov. **Progressive Cramming: Reliable Token Compression and What It Reveals**. arXiv:2607.21231, July 23, 2026. <https://arxiv.org/abs/2607.21231>